

15. Design of Image Intensity Inversion Circuit Using Xilinx Vivado HLS

Aim

To design and implement an image intensity inversion system using Xilinx Vivado HLS, where a C++ based image inversion algorithm is converted into RTL hardware, verified using a Verilog testbench, and the processed image output is visualized using Python.

Software Required

- Xilinx Vivado Design Suite
- Vivado HLS
- Vivado Simulator (XSim)
- Verilog HDL
- Python

Procedure

C++ Algorithm Development

- A C++ function was written using `ap_uint<8>` to perform image intensity inversion.
- For each grayscale pixel, the output pixel value was computed as $255 - \text{pixel_in}$.

High-Level Synthesis

- The C++ program was synthesized using Vivado HLS.
- `ap_none` interfaces were used to generate a simple data-only RTL design.
- Pipelining was applied to improve processing throughput.

RTL Generation

- Vivado HLS converted the C++ design into Verilog RTL.
- The generated RTL implemented a subtraction-based image inversion circuit.

Verilog Testbench Verification

- A Verilog testbench was written to apply pixel values to the RTL module.
- The inverted pixel values were captured during simulation.

Image Input Generation

- A grayscale image was converted into a pixel sequence using Python.
- The pixel sequence was provided as input to the Verilog testbench.

Output Visualization

- The simulation output was written to a text file.
- Python was used to reconstruct and display the inverted image.

Python program to convert the 256x256 image to 1D form suitable for verilog module

```
from PIL import Image

import numpy as np

img = Image.open("input_image.png").convert("L")

img = img.resize((256, 256))

img_array = np.array(img, dtype=np.uint8)

with open("image_in.txt", "w") as f:

    for pixel in img_array.flatten():

        f.write(str(int(pixel)) + "\n")
```

C++ program for intensity inversion

```
#include <ap_int.h>

ap_uint<8> intensity_inversion(ap_uint<8> pixel_in) {

#pragma HLS INTERFACE ap_none port=pixel_in

#pragma HLS INTERFACE ap_none port=return

#pragma HLS PIPELINE

    ap_uint<8> pixel_out;

    pixel_out = 255 - pixel_in;

    return pixel_out;

}
```

Testbench program

```
`timescale 1ns/1ps

module tb_image_threshold;

    parameter WIDTH = 256;

    parameter HEIGHT = 256;

    parameter SIZE = WIDTH * HEIGHT;

    reg [7:0] gray_pixel;

    wire [7:0] binary_pixel;

    reg [7:0] image_mem [0:SIZE-1];

    integer i;

    integer infile, outfile;
```

```

image_threshold UUT (
    .ap_start(1'b1),
    .ap_done(),
    .ap_idle(),
    .ap_ready(),
    .pixel_in_V(gray_pixel),
    .ap_return(binary_pixel)
);

initial begin
    infile = $fopen("I:\\Saveetha\\SOC\\Lab experiments\\image_in.txt", "r");
    outfile = $fopen("I:\\Saveetha\\SOC\\Lab experiments\\image_out.txt", "w");
    for (i = 0; i < SIZE; i = i + 1) begin
        if (!$feof(infile))
            $fscanf(infile, "%d\\n", image_mem[i]);
    end
    for (i = 0; i < SIZE; i = i + 1) begin
        gray_pixel = image_mem[i];
        #1;
        $fdisplay(outfile, "%d", binary_pixel);
    end
    $fclose(infile);
    $fclose(outfile);
    $finish;
end
endmodule

```

Python program to convert the 1D output data generated by verilog module to 256x256 image

```

import numpy as np
from PIL import Image
data = np.loadtxt("image_out.txt", dtype=np.uint8)
img = data.reshape((256,256))

```

```
Image.fromarray(img).save("inverted_image.png")
```

Output:



Grayscale image



Intensity inverted image

Result

The image intensity inversion system was successfully designed and verified. The RTL generated from the C++ program correctly inverted the grayscale image by mapping low-intensity pixels to high intensity and vice versa. The output image reconstructed using Python clearly demonstrated correct inversion, confirming the proper functionality of the HLS-based design.